Although this is quite a long function, it's actually straightforward. If the input range is empty ❶, you just return the initial value `init`. Otherwise, there's at least one element in the range, so you can divide the number of elements to process by the minimum block size in order to give the maximum number of threads ❷. This is to avoid creating 32 threads on a 32-core machine when you have only five values in the range.

The number of threads to run is the minimum of your calculated maximum and the number of hardware threads ❸. You don't want to run more threads than the hardware can support (which is called *oversubscription*), because the context switching will mean that more threads will decrease the performance. If the call to `std::thread::hardware_concurrency()` returned 0, you'd simply substitute a number of your choice; in this case I've chosen 2. You don't want to run too many threads, because that would slow things down on a single-core machine, but likewise you don't want to run too few, because then you'd be passing up the available concurrency.

The number of entries for each thread to process is the length of the range divided by the number of threads ❹. If you're worrying about the case where the number doesn't divide evenly, don't—you'll handle that later.

Now that you know how many threads you have, you can create a `std::vector<T>` for the intermediate results and a `std::vector<std::thread>` for the threads ❺. Note that you need to launch one fewer thread than `num_threads`, because you already have one.

Launching the threads is just a simple loop: advance the `block_end` iterator to the end of the current block ❻ and launch a new thread to accumulate the results for this block ❼. The start of the next block is the end of this one ❽.

After you've launched all the threads, this thread can then process the final block ❾. This is where you take account of any uneven division: you know